

Guia de estudo

Case técnico de Python - preparação para conversa com o CEO

Como usar

Leia as quatro primeiras páginas para entender a lógica. Depois pratique as respostas de entrevista em voz alta. A última página é uma revisão de 2 minutos.

O que esta case avalia

Ponto	Problema	Habilidade
1	Somar dados de um JSON imperfeito	Qualidade, validação e conversão de dados
2	Comparar textos em codificações diferentes	Bytes, codificação e comparação
3 a 6	Calcular dias entre datas	Biblioteca padrão e calendário

A ideia central

A case não mede apenas se você chega ao número final. Ela mede se entende a natureza do dado antes de processá-lo. Um dado pode falhar por formato, tipo, codificação ou regra de calendário. A sequência correta é: ler, normalizar, validar e só então calcular ou comparar.

Vocabulário mínimo

- Variável: nome que guarda um valor, como ano.
- Função: bloco reutilizável, como `calcular_dias(ano)`.
- JSON: formato de texto usado para troca de dados entre sistemas.
- Exceção: erro previsto e tratado para o programa não falhar sem explicação.

1. Estrutura da solução

O README separa a entrega em três programas pequenos. Isso mantém cada responsabilidade isolada, facilita testes e torna a explicação mais clara.

Arquivo	Função	Resultado
ponto1.py	Lê o JSON, corrige aspas e soma valores.	10
ponto2.py	Lê bytes, decodifica e compara textos.	mesmo conteúdo
pontos3_a_6.py	Monta datas e calcula a diferença.	243, 242, 242, 242

Como comunicar a arquitetura

Resposta curta

Eu dividi os problemas por responsabilidade. Cada arquivo trata um tipo de dado e usa a biblioteca padrão adequada. Isso deixa a solução simples, verificável e fácil de explicar.

2. Ponto 1 - JSON e soma

A entrada é `[1, "2", 3, "4"]`. A resposta é 10, mas há dois problemas: as aspas são tipográficas e dois números estão como texto. JSON aceita aspas retas, não aspas tipográficas.

```
texto = caminho.read_text(encoding="utf-8-sig")
texto = texto.replace("\"", "'').replace("'", "'")
dados = json.loads(texto)
numeros = [int(valor) for valor in dados]
return sum(numeros)
```

Linha a linha

- `read_text` lê o arquivo. `utf-8-sig` também tolera um marcador invisível inicial.
- `replace` troca caracteres inválidos pelas aspas aceitas pelo JSON.
- `json.loads` transforma texto em uma lista Python.
- `int(valor)` converte "2" em 2; texto e número não são o mesmo tipo.
- `sum` agrega $1 + 2 + 3 + 4 = 10$.

Como defender o ponto 1

Resposta-modelo

Antes de calcular, eu garanto que o dado esteja em um formato confiável. A solução corrige o problema conhecido de formatação, valida que a estrutura é uma lista e converte os valores para o tipo numérico antes de somar.

3. Ponto 2 - textos, bytes e codificação

O conceito-chave é: bytes não são texto. O arquivo guarda bytes; eles só representam letras quando aplicamos uma codificação. Os arquivos têm bytes diferentes, mas viram o mesmo texto após decodificação correta.

Arquivo	Caractere final	Bytes	Codificação
test2_text1.txt	è!?	C3 A8 21 3F	UTF-8
test2_text2.txt	è!?	E8 21 3F	Windows-1252 / Latin-1

```
dados = caminho.read_bytes()
for encoding in ("utf-8", "cp1252", "latin-1"):
    try:
        return dados.decode(encoding)
    except UnicodeDecodeError:
        continue
```

Linha a linha

- `read_bytes` lê o conteúdo bruto, sem assumir codificação.
- O código tenta UTF-8, depois cp1252 (comum no Windows) e Latin-1.
- `UnicodeDecodeError` é o erro esperado se aqueles bytes não pertencem à codificação tentada.
- `texto1 == texto2` verifica texto igual e na mesma ordem.
- `Counter(texto1) == Counter(texto2)` verifica que cada caractere, espaço e pontuação aparece a mesma quantidade de vezes.

Como defender

Eu não compararia bytes diretamente: o mesmo conteúdo pode ser armazenado de maneiras diferentes. Primeiro converto ambos para uma representação textual comum; só então comparo o significado do conteúdo.

4. Pontos 3 a 6 - datas e anos bissextos

O programa cria 15 de fevereiro e 15 de outubro de cada ano. A subtração de datas é delegada à biblioteca padrão do Python.

```
inicio = date(ano, 2, 15)
fim = date(ano, 10, 15)
return (fim - inicio).days
```

O resultado é a quantidade de dias transcorridos entre as datas. O dia inicial não conta como dia completo transcorrido.

Ano	Dias	Explicação
2000	243	É bissexto; 29 de fevereiro existe no intervalo.
2001	242	Não é divisível por 4.
1900	242	É divisível por 100, mas não por 400.
1582	242	Python usa calendário gregoriano contínuo.

Regra de ano bissexto

- Divisível por 4: normalmente é bissexto.
- Divisível por 100: deixa de ser bissexto.
- Divisível por 400: volta a ser bissexto.

O contraste entre 2000 e 1900 foi incluído para testar a exceção dos anos centenários.

Atenção ao ano 1582

Historicamente, os países adotaram o calendário gregoriano em momentos diferentes. Python não modela essa transição local: `datetime` usa calendário gregoriano proléptico. Para o teste, essa é a convenção correta; num produto histórico, a premissa precisaria ser discutida.

5. Simulação de entrevista

Pratique as respostas em voz alta. O objetivo é mostrar lógica, clareza e consciência dos limites da solução.

Qual foi sua abordagem geral?

Resposta-modelo

Eu dividi a entrega por tipo de problema. No primeiro caso tratei formato e tipo de dado; no segundo, codificação de texto; no terceiro, regras de data. Em todos, a sequência foi normalizar, validar e calcular usando a biblioteca padrão.

Por que não usar apenas uma soma no JSON?

Resposta-modelo

Porque o arquivo não era JSON válido e misturava números com textos. Somar diretamente poderia falhar ou esconder um problema de qualidade. Primeiro corrijo a estrutura, faço o parsing e converto explicitamente os valores.

Como dois arquivos têm o mesmo texto e bytes diferentes?

Resposta-modelo

A codificação define como um caractere vira bytes. O `o` ocupa dois bytes em UTF-8 e um byte em Windows-1252. Depois da decodificação correta, o texto representado é o mesmo.

Por que usar Counter além de ==?

Resposta-modelo

A igualdade verifica texto e ordem. Counter verifica uma hipótese diferente: se a frequência de cada caractere é igual, sem depender da ordem. Na case os dois testes retornam verdadeiro.

O que o caso de 1900 prova?

Resposta-modelo

Que não basta ser múltiplo de quatro. Anos múltiplos de cem só são bissextos se também forem múltiplos de quatrocentos. Por isso 1900 não adiciona 29 de fevereiro.

6. Perguntas de aprofundamento

Pergunta	Bom resposta
Que melhoria faria em produção?	Definiria UTF-8 como contrato de integração, registraria desvios e validaria tipos do JSON mais estritamente.
A troca de aspas sempre é segura?	É adequada porque o exercício revela um problema conhecido. Em produção, eu limitaria a normalização ao contexto esperado e registraria a alteração.
Counter ignora espaços?	Não. Inclui espaços, pontuação e quebras de linha.
Python usa o calendário histórico de 1582?	Não exatamente. datetime usa calendário gregoriano proléptico. A resposta depende da convenção da biblioteca.
Por que usar biblioteca padrão?	Ela é testada, reduz erro manual e deve ser preferida quando cobre a regra de negócio.

Erros que você deve evitar

- Não diga que os arquivos eram idênticos em bytes. Eles não eram.
- Não diga que todo ano múltiplo de 4 é bissexto. 1900 é o contraexemplo.
- Não chame "2" de número. É texto que representa um número.
- Não afirme que 1582 reproduz a história de todos os países.

Frase de maturidade

Em produção, eu deixaria as premissas explícitas e criaria testes para entradas inválidas, codificações inesperadas e datas de fronteira. A resposta correta depende tanto da regra de negócio quanto do algoritmo.

Revisão final - 2 minutos

A narrativa em 30 segundos

A case trata três fontes comuns de erro: formato de dados, codificação de texto e regra de calendário.

Eu normalizei e validei os dados antes de operá-los, usei bibliotecas padrão e deixei explícitas as premissas.

Resultados para memorizar

Ponto	Resposta	Por quê
JSON	10	Corrige aspas, faz parsing, converte textos numéricos e soma.
Textos	Iguais após decodificação	UTF-8 e CP1252 guardam <code>è</code> em bytes diferentes.
2000	243 dias	É bissexto.
2001	242 dias	Não é bissexto.
1900	242 dias	Século não divisível por 400.
1582	242 dias	Calendário contínuo do Python.

Frases para lembrar

- Formato primeiro, cálculo depois.
- Bytes não são texto; codificação dá significado aos bytes.
- Texto e número são tipos diferentes.
- Explique premissas, não apenas resultados.

Fechamento sugerido

Resposta final

O principal aprendizado é que qualidade de dados não é uma etapa separada do problema: ela faz parte da solução. Antes de somar, comparar ou calcular datas, é preciso garantir que estamos interpretando corretamente o dado que recebemos.